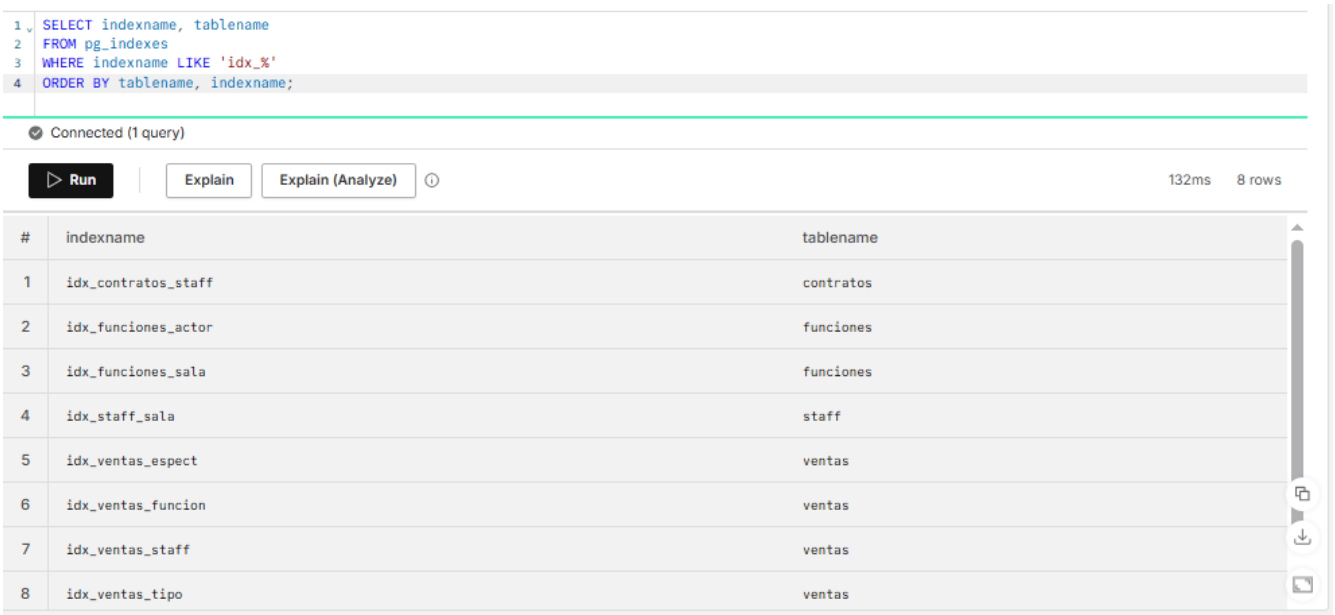
Evidencias de las correcciones del Avance 1

Bosque Cinema · Bases de Datos 2 · Universidad El Bosque

Capturas de la ejecución en Neon (PostgreSQL) de los scripts corregidos del Avance 1: índices sobre las llaves foráneas (01), carga de datos re-ejecutable (02), control de aforo por capacidad de la sala (04), rankings con NULLS LAST (05) y pruebas de los triggers y procedimientos. Cada prueba de trigger se ejecutó dentro de BEGIN/ROLLBACK para no alterar los datos.



```
SELECT indexname, tablename
FROM pg_indexes
WHERE indexname LIKE 'idx_%'
ORDER BY tablename, indexname;
```

Connected (1 query)

Run | Explain | Explain (Analyze) — 132ms — 8 rows

#	indexname	tablename
1	idx_contratos_staff	contratos
2	idx_funciones_actor	funciones
3	idx_funciones_sala	funciones
4	idx_staff_sala	staff
5	idx_ventas_espect	ventas
6	idx_ventas_funcion	ventas
7	idx_ventas_staff	ventas
8	idx_ventas_tipo	ventas

Evidencia 1. Índices creados en PostgreSQL (script 01 corregido).



```
BEGIN;

-- Vacía las tablas y reinicia los id para poder ejecutar el script varias veces
TRUNCATE ventas, contratos, funciones, staff, espectadores, actores_directores, tipos_boleta, salas
    RESTART IDENTITY CASCADE;

-- 1. Salas (5)
INSERT INTO salas (nombre, capacidad, tipo_sala, estado) VALUES
('Roble',     120, '2D',  'ACTIVA'),   -- id 1
('Cedro',     100, '3D',  'ACTIVA'),   -- id 2
('Guayacán',   40, 'VIP', 'ACTIVA'),   -- id 3
('Ceiba',     150, '2D',  'ACTIVA'),   -- id 4
('Sauce',      90, '3D',  'ACTIVA');   -- id 5

-- 2. Tipos_Boleta (5)
INSERT INTO tipos_boleta (nombre, descripcion, precio, activo) VALUES
('General',          'Tarifa estándar para público en general',          18000.00, TRUE),  -- 1
```

Connected (11 queries) · This query will still run OK, but the last 19452 characters will be truncated from query history

Run | Explain | Explain (Analyze) — 135ms — No result

1: BEGIN · 2: TRUNCATE · 3: INSERT 5 · 4: INSERT 5 · 5: INSERT 32 · 6: INSERT 32 · 7: INSERT 30 · 8: INSERT 30 · 9: INSERT 32 · 10: INSERT 45 · 11: COMMIT

Statement executed successfully

Evidencia 2. Ejecución de 02_datos_prueba.sql corregido (TRUNCATE ... RESTART IDENTITY), sin errores.

```sql
-- CONSULTA 4
-- Tablas: funciones, salas, ventas
-- Recaudo total por función con analítica avanzada:
-- RANK() particionado por formato (2D, 3D, etc.) y
-- ROW_NUMBER() para el ranking general de taquilla.

SELECT
    f.id_funcion,
    f.titulo_pelicula,
    f.formato,
    s.nombre,
    COALESCE(SUM(v.cantidad * v.precio_unitario), 0) AS recaudo_total,
    RANK() OVER (PARTITION BY f.formato ORDER BY SUM(v.cantidad * v.precio_unitario) DESC NULLS LAST) AS ranking_formato,
    ROW_NUMBER() OVER (ORDER BY SUM(v.cantidad * v.precio_unitario) DESC NULLS LAST) AS ranking_general
FROM funciones f
INNER JOIN salas s ON f.id_sala = s.id_sala
LEFT JOIN ventas v ON f.id_funcion = v.id_funcion AND v.estado = 'PAGADA'
GROUP BY f.id_funcion, f.titulo_pelicula, f.formato, s.nombre
ORDER BY recaudo_total DESC;
```

Connected (1 query)

Run | Explain | Explain (Analyze) — 162ms · 32 rows

#	id_funcion	titulo_pelicula	formato	nombre	recaudo_total	ranking_formato	ranking_general
1	7	El Jardín de Cristal	2D	Guayacán	134000.00	1	1
2	4	Galaxia Nómada	3D	Cedro	116000.00	1	2
3	13	Rumbo a Marte	3D	Sauce	100000.00	2	3
4	1	La Ruta del Colibrí	2D	Roble	92000.00	2	4
5	5	Océano Profundo	3D	Cedro	88000.00	3	5
6	8	Tormenta Roja	2D	Guayacán	86000.00	3	6

Evidencia 3. Consulta 4 corregida y su resultado.

```sql
-- TRIGGER 1: trg_validar_desactivar_sala
-- No se puede desactivar una sala con funciones programadas a futuro.
BEGIN;
INSERT INTO funciones (id_sala, id_actor_director, titulo_pelicula, clasificacion, formato, version_idioma, fecha_hora_inicio, fecha_hora_fin, estado)
VALUES (1, 3, 'La Ruta del Colibrí', 'T', '2D', 'DOBLADA', CURRENT_DATE + 30 + TIME '15:00', CURRENT_DATE + 30 + TIME '16:35', 'PROGRAMADA');
UPDATE salas SET estado = 'INACTIVA' WHERE id_sala = 1;
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN | 2: INSERT 1 | 3: ERROR

ERROR: No se puede desactivar la sala 1, porque tiene funciones programadas o en curso. (SQLSTATE P0001) — Fix with AI

Evidencia 4. Prueba del trigger trg_validar_desactivar_sala.

```sql
-- TRIGGER 2: trg_validar_desactivar_actor_director
-- No se puede desactivar un actor/director con funciones programadas a futuro.
BEGIN;
INSERT INTO funciones (id_sala, id_actor_director, titulo_pelicula, clasificacion, formato, version_idioma, fecha_hora_inicio, fecha_hora_fin, estado)
VALUES (1, 3, 'La Ruta del Colibrí', 'T', '2D', 'DOBLADA', CURRENT_DATE + 30 + TIME '15:00', CURRENT_DATE + 30 + TIME '16:35', 'PROGRAMADA');
UPDATE actores_directores SET activo = FALSE WHERE id_actor_director = 3;
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN | 2: INSERT 1 | 3: ERROR

ERROR: No se puede desactivar el actor/director 3, porque tiene funciones programadas o en curso. (SQLSTATE P0001) — Fix with AI

Evidencia 5. Prueba del trigger trg_validar_desactivar_actor_director.

```sql
-- TRIGGER 3: trg_validar_contrato
-- Un staff no puede tener dos contratos vigentes en el mismo periodo.
BEGIN;
INSERT INTO contratos (id_staff, tipo_contrato, jornada, fecha_inicio, fecha_fin, salario_mensual, estado)
VALUES (1, 'INDEFINIDO', 'COMPLETA', CURRENT_DATE, NULL, 3000000, 'VIGENTE');
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN 2: ERROR

ERROR: El staff 1 ya tiene otro contrato vigente en el mismo periodo. (SQLSTATE P0001) Fix with AI

Evidencia 6. Prueba del trigger trg_validar_contrato.

```sql
-- TRIGGER 4: trg_validar_desactivar_staff
-- No se puede desactivar un staff con contrato vigente.
BEGIN;
UPDATE staff SET activo = FALSE WHERE id_staff = 1;
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN 2: ERROR

ERROR: No se puede desactivar el staff 1, porque tiene un contrato vigente. (SQLSTATE P0001) Fix with AI

Evidencia 7. Prueba del trigger trg_validar_desactivar_staff.

```sql
-- TRIGGER 5: trg_validar_fecha_espectador
-- La fecha de nacimiento no puede ser posterior a la fecha de registro.
BEGIN;
UPDATE espectadores SET fecha_nacimiento = fecha_registro + 1 WHERE id_espectador = 1;
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN 2: ERROR

ERROR: La fecha de nacimiento no puede ser posterior a la fecha de registro. (SQLSTATE P0001) Fix with AI

Evidencia 8. Prueba del trigger trg_validar_fecha_espectador.

```sql
-- TRIGGER 6: trg_validar_precio_boleta
-- No se puede cambiar el precio de un tipo de boleta inactivo.
BEGIN;
UPDATE tipos_boleta SET activo = FALSE WHERE id_tipo_boleta = 5;
UPDATE tipos_boleta SET precio = 15000 WHERE id_tipo_boleta = 5;
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN 2: UPDATE 1 3: ERROR

ERROR: No se puede modificar el precio del tipo de boleta 5 porque está inactivo. (SQLSTATE P0001) Fix with AI

Evidencia 9. Prueba del trigger trg_validar_precio_boleta.

```sql
-- TRIGGER 7: trg_validar_horario_funcion
-- Una sala no puede tener dos funciones al mismo tiempo (choca con la función 1).
BEGIN;
INSERT INTO funciones (id_sala, id_actor_director, titulo_pelicula, clasificacion, formato, version_idioma, fecha_hora_inicio, fecha_hora_fin, estado)
SELECT id_sala, 5, 'Noche en Villa Esperanza', '+15', '2D', 'SUBTITULADA', fecha_hora_inicio + INTERVAL '30 minutes', fecha_hora_fin + INTERVAL '30 minutes', 'PROGRAMADA'
FROM funciones WHERE id_funcion = 1;
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN 2: ERROR

ERROR: La sala 1 ya tiene otra función programada en ese horario. (SQLSTATE P0001) Fix with AI

Evidencia 10. Prueba del trigger trg_validar_horario_funcion.

```sql
-- TRIGGER 8: trg_validar_capacidad_venta
-- No se pueden vender más boletas que la capacidad de la sala.
-- Función 9, sala Guayacán (capacidad 40), ya tiene 5 boletas vendidas.
BEGIN;
INSERT INTO ventas (id_espectador, id_funcion, id_tipo_boleta, id_staff, fecha_venta, cantidad, precio_unitario, metodo_pago, estado) VALUES
(1, 9, 1, 3, '2026-10-03 10:00', 10, 18000.00, 'TARJETA', 'PAGADA'),
(2, 9, 1, 3, '2026-10-03 10:05', 10, 18000.00, 'TARJETA', 'PAGADA'),
(4, 9, 1, 3, '2026-10-03 10:10', 10, 18000.00, 'TARJETA', 'PAGADA');   -- 35 de 40
INSERT INTO ventas (id_espectador, id_funcion, id_tipo_boleta, id_staff, fecha_venta, cantidad, precio_unitario, metodo_pago, estado)
VALUES (5, 9, 1, 3, '2026-10-03 10:15', 10, 18000.00, 'TARJETA', 'PAGADA');   -- 45 de 40: error
ROLLBACK;
```

Failed transaction: ROLLBACK required

ROLLBACK

1: BEGIN 2: INSERT 3 3: ERROR

ERROR: No se puede realizar la venta. La capacidad de la sala es 40 y se superarían las boletas permitidas. (SQLSTATE P0001) Fix with AI

Evidencia 11. Prueba del trigger trg_validar_capacidad_venta.

```sql
CALL sp_calcular_ocupacion(9, NULL, NULL, NULL);
```

Connected (1 query)

Run | Explain | Explain (Analyze) 122ms 1 row

#	p_vendidas	p_disponibles	p_recaudo
1	5	35	75000.00

Evidencia 12. Ejecución del procedimiento sp_calcular_ocupacion.

```sql
CALL sp_reporte_ventas_pelicula(NULL, NULL, NULL);
```

Connected (1 query)

Run | Explain | Explain (Analyze) 220ms 1 row

#	p_total_boletas	p_total_recaudado	p_pelicula_top
1	101	1474000.00	Rumbo a Marte

Evidencia 13. Ejecución del procedimiento sp_reporte_ventas_pelicula.

```
BEGIN;
CALL sp_anular_venta(1, NULL);
ROLLBACK;
```

Connected (3 queries)

Run | Explain | Explain (Analyze) — 154ms — 1 row

1: BEGIN | 2: CALL | 3: ROLLBACK

#	p_valor_devuelto
1	20000.00

Evidencia 14. Ejecución del procedimiento sp_anular_venta.

```
BEGIN;
CALL sp_registrar_venta(1, 20, 1, 3, 2, 'TARJETA', NULL, NULL);
ROLLBACK;
```

Connected (3 queries)

Run | Explain | Explain (Analyze) — 125ms — 1 row

1: BEGIN | 2: CALL | 3: ROLLBACK

#	p_id_venta	p_total
1	50	36000.00

Evidencia 15. Ejecución del procedimiento sp_registrar_venta.

```
-- CONSULTA 1
-- Tablas: ventas
-- Agrupa las ventas efectivas (PAGADA) por método de pago
-- para identificar los medios de pago más usados y el
-- volumen de dinero que ingresa por cada canal.

SELECT
    metodo_pago,
    COUNT(id_venta) AS total_transacciones,
    SUM(cantidad) AS total_boletas_vendidas,
    SUM(cantidad * precio_unitario) AS recaudo_total
FROM ventas
WHERE estado = 'PAGADA'
GROUP BY metodo_pago
ORDER BY recaudo_total DESC;
```

Connected (1 query)

Run | Explain | Explain (Analyze) — 165ms — 3 rows

#	metodo_pago	total_transacciones	total_boletas_vendidas	recaudo_total
1	TARJETA	20	51	847000.00
2	EFECTIVO	17	34	400000.00
3	TRANSFERENCIA	7	16	227000.00

Evidencia 16. Consulta 1 del Avance 1 y su resultado.

```sql
-- CONSULTA 2
-- Tablas: funciones, salas
-- INNER JOIN entre las funciones programadas y las salas
-- para obtener la cartelera activa con la capacidad de
-- cada sala.

SELECT
    f.id_funcion,
    f.titulo_pelicula,
    f.clasificacion,
    f.formato,
    s.nombre,
    s.capacidad AS capacidad_sala,
    f.fecha_hora_inicio,
    f.estado AS estado_funcion
FROM funciones f
INNER JOIN salas s ON f.id_sala = s.id_sala
WHERE f.estado = 'PROGRAMADA'
ORDER BY f.fecha_hora_inicio ASC;
```

Connected (1 query)

Run | Explain | Explain (Analyze) — 161ms — 16 rows

#	id_funcion	titulo_pelicula	clasificacion	formato	nombre	capacidad_sala	fecha_hora_inicio	estado_funcion
1	17	Latidos	+12	2D	Roble	120	2026-10-07 17:00:00	PROGRAMADA
2	18	Duelo en la Frontera	+15	2D	Roble	120	2026-10-07 20:00:00	PROGRAMADA
3	21	Rumbo a Marte	+7	3D	Cedro	100	2026-10-08 17:30:00	PROGRAMADA
4	22	Océano Profundo	+12	3D	Cedro	100	2026-10-08 20:00:00	PROGRAMADA
5	24	Tormenta Roja	+18	2D	Guayacán	40	2026-10-09 20:30:00	PROGRAMADA
6	27	El Silencio del Valle	+18	2D	Ceiba	150	2026-10-09 21:00:00	PROGRAMADA
7	19	La Ruta del Colibrí	T	2D	Roble	120	2026-10-10 14:00:00	PROGRAMADA

Evidencia 17. Consulta 2 del Avance 1 y su resultado.

```sql
-- CONSULTA 3
-- Tablas: espectadores, ventas
-- Identifica a los clientes cuyas compras superan el
-- promedio general de boletas por transacción (subconsulta).
-- Útil para fidelización y campañas de marketing.

SELECT
    e.id_espectador,
    e.nombres || ' ' || e.apellidos AS nombre_completo,
    e.correo,
    v.id_venta,
    v.cantidad,
    v.precio_unitario
FROM espectadores e
INNER JOIN ventas v ON e.id_espectador = v.id_espectador
WHERE v.estado = 'PAGADA'
  AND v.cantidad > (
      SELECT AVG(cantidad)
      FROM ventas
      WHERE estado = 'PAGADA'
  )
ORDER BY v.cantidad DESC;
```

Connected (1 query)

Run | Explain | Explain (Analyze) — 169ms — 12 rows

#	id_espectador	nombre_completo	correo	id_venta	cantidad	precio_unitario
1	6	Sebastián Ramírez Cruz	sebastian.ramirez@correo.example	2	4	18000.00
2	8	Andrés Castro Villamil	andres.castro@correo.example	8	4	18000.00
3	28	Esteban Murillo Gaitán	esteban.murillo@correo.example	14	4	13000.00
4	2	Santiago Rojas Mejía	santiago.rojas@correo.example	17	4	18000.00
5	5	Isabella Torres Quintero	isabella.torres@correo.example	30	4	14000.00

Evidencia 18. Consulta 3 del Avance 1 y su resultado.